Arquivos

Prof. Thiago Felski Pereira, MSc.

Por que arquivos?

- Toda vez que um programa termina, as variáveis que ele criou desaparecem da memória.
- Se você quer que os dados sobrevivam entre execuções — uma lista de alunos, um histórico, uma configuração — eles precisam ser gravados em algum lugar permanente: um arquivo.
- **É exatamente isso que a biblioteca fstream nos permite fazer: ler e escrever arquivos no disco, do mesmo jeito que já lemos e escrevemos no console.**

UNIVALI

Biblioteca fstream

```
#include <fstream>
```

- A biblioteca fstream fornece classes que permitem realizar leitura e escrita em arquivos, de maneira parecida com o cin e o cout que já usamos para leitura e escrita no console.
- A única diferença é que precisamos associar nossos streams a arquivos físicos.

UNIVALI

Três classes para arquivos

`ofstream`	classe stream para escrita em arquivos
`ifstream`	classe stream para leitura de arquivos
`fstream`	classe stream para leitura e escrita em arquivos

UNIVALI

Escrevendo em um arquivo: os 4 passos

PASSO 1: crie um stream do tipo ofstream

```
ofstream meu_arquivo;
```

PASSO 2: associe um arquivo ao seu stream

```
meu_arquivo.open("exemplo.txt");
```

PASSO 3: envie dados para esse stream

```
meu_arquivo << "Escrevendo nesse arquivo.\n";
```

PASSO 4: encerre a utilização do arquivo

```
meu_arquivo.close();
```


UNIVALI

Escrevendo em um arquivo: exemplo

```
#include <fstream>
using namespace std;

int main () {
    ofstream meu_arquivo; // PASSO 1
    meu_arquivo.open ("exemplo.txt"); // PASSO 2
    meu_arquivo << "Escrevendo nesse arquivo.\n"; // PASSO 3
    meu_arquivo.close(); // PASSO 4
    return 0;
}
```


UNIVALI

Escrevendo outros tipos de dados

- Podemos escrever outras coisas além de strings prontas — inteiros, floats, caracteres, variáveis.

```
float valor = 999;
ofstream meu_arquivo;
meu_arquivo.open ("exemplo.txt");
meu_arquivo << 10 << "\n";
meu_arquivo << 5.45 << "\n";
meu_arquivo << 'x' << "\n";
meu_arquivo << valor << "\n";
meu_arquivo.close();
```

conteúdo de exemplo.txt:

```
10
5.45
x
999
```


UNIVALI

Praticando escrita em arquivos

ATIVIDADE 1

- **Faça um programa que armazene os dados de 5 alunos de uma turma em um arquivo.**

UNIVALI

Praticando escrita em arquivos

ATIVIDADE 2

- **Faça uma função em que o usuário fornece, como parâmetros, os valores de A, B e C de uma equação do segundo grau, e salva em arquivo a equação e o seu resultado.**
- Por exemplo, se o usuário forneceu A=1, B=-5, C=6, o arquivo deverá conter:
  - $1x^2 - 5x + 6 = 0$
  - $x' = 3$ e $x'' = 2$

UNIVALI

Verificando se o arquivo abriu corretamente

- Em muitos programas é necessário ter certeza de que o arquivo foi corretamente aberto antes de tentar executar qualquer operação.
- No fstream, esse comando chama-se:

```
meu_arquivo.is_open();
```


UNIVALI

Exemplo: verificação de abertura

```
ofstream meu_arquivo;
meu_arquivo.open ("exemplo.txt");
if ( meu_arquivo.is_open() ) {
    meu_arquivo << "Salvando no arquivo aberto";
    meu_arquivo.close();
} else {
    cout << "O arquivo não pode ser aberto";
}
```


UNIVALI

Lendo de um arquivo: os 4 passos

PASSO 1: crie um stream do tipo ifstream

```
ifstream meu_arquivo;
```

PASSO 2: associe um arquivo ao seu stream

```
meu_arquivo.open("exemplo.txt");
```

PASSO 3: leia dados desse stream (um caractere por vez)

```
char lido;
lido = meu_arquivo.get();
```

PASSO 4: encerre a utilização do arquivo

```
meu_arquivo.close();
```


UNIVALI

Lendo de um arquivo: exemplo

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
    char lido;
    ifstream meu_arquivo; // PASSO 1
    meu_arquivo.open ("exemplo.txt"); // PASSO 2
    lido = meu_arquivo.get(); // PASSO 3
    meu_arquivo.close(); // PASSO 4
    return 0;
}
```


UNIVALI

Lendo palavras, linhas, ou o arquivo inteiro

- Ler uma palavra inteira — fica parecido com o cin:

```
string palavra;
meu_arquivo >> palavra;
```

- Ler uma linha inteira:

```
string frase;
getline(meu_arquivo, frase);
```

- Ler o arquivo inteiro:

```
string frase;
while (getline(meu_arquivo, frase)) {
    cout << frase << "\n";
}
```

Atenção: evite while (meu_arquivo.eof()==false) — eof() só fica verdadeiro depois de uma leitura falhar, o que pode duplicar a última linha. Colocar a leitura direto na condição do laço, como acima, evita esse erro clássico.

UNIVALI

Praticando leitura em arquivos

ATIVIDADE 3

- **Faça um programa que, caso o arquivo tenha sido corretamente aberto, leia ele inteiro e imprima no console.**

UNIVALI

Praticando leitura em arquivos

ATIVIDADE 4

- **Faça um programa que:**
  - carregue vários nomes e idades em um arquivo;
  - permita procurar por um nome e, caso encontre, imprima a idade daquela pessoa.

UNIVALI

Praticando leitura em arquivos

ATIVIDADE 5

- **Leia uma matriz que está em um arquivo, e carregue-a em uma matriz na memória.**

UNIVALI

Arquivos CSV

- CSV (Comma-Separated Values) é um arquivo que separa os dados por vírgulas.
- É comumente usado no Microsoft Excel e no LibreOffice Calc.

```
1997,Ford,E350,"ac, abs, moon",30100.00
1999,Chevy,"Venture ""Extended Edition""",,49000.00
1996,Jeep,Grand Cherokee,"MUST SELL!
air, moon roof, loaded",479699.00
```


UNIVALI

Escrevendo em um arquivo CSV

- Siga os mesmos 4 passos de uma escrita normal:

PASSO 1: crie um stream do tipo ofstream

```
ofstream meu_arquivo;
```

PASSO 2: associe um arquivo ao seu stream

```
meu_arquivo.open("tabela.csv");
```

PASSO 3: envie dados para esse stream

```
meu_arquivo << "Escrevendo nesse arquivo.\n";
```

PASSO 4: encerre a utilização do arquivo

```
meu_arquivo.close();
```


UNIVALI

Escrevendo em uma linha ou coluna única

- Escrevendo em uma única linha:

```
meu_arquivo << "sep=,\n";
for (int i = 0; i < 10; i++) {
    meu_arquivo << i << ',';
}
```

- Escrevendo em uma única coluna:

```
for (int i = 0; i < 10; i++) {
    meu_arquivo << i << "\n";
}
```

- Para o Excel abrir corretamente, indique na 1ª linha o separador de colunas usado: sep=,

UNIVALI

Escrevendo fórmulas do Excel

- Você também pode escrever comandos do Excel dessa forma:

```
meu_arquivo << "=SOMA(A1:A10)";
```

```
int main () {
    ofstream meu_arquivo;
    meu_arquivo.open("tabela.csv");
    for (int i = 0; i < 10; i++) {
        meu_arquivo << i << "\n";
    }
    meu_arquivo << "=SOMA(A1:A10)";
    meu_arquivo.close();
    return 0;
}
```


UNIVALI

Lendo de um arquivo CSV

- A leitura de um arquivo CSV acontece da mesma forma que qualquer outro arquivo de texto.

```
ifstream meu_arquivo;
meu_arquivo.open("tabela.csv");
char lido;
lido = meu_arquivo.get();
meu_arquivo.close();
```

Atenção: get() lê todos os caracteres, incluindo a linha sep=,, as vírgulas e o caractere de fim de linha — nada disso é filtrado automaticamente.

UNIVALI

Separando os campos de uma linha CSV

- Ler o arquivo inteiro é só o primeiro passo — normalmente você quer separar cada linha nos seus campos individuais.
- Uma forma simples é usar um stringstream junto com getline, parando em cada vírgula em vez de no fim da linha:

```
#include <sstream>

string linha, campo;
getline(meu_arquivo, linha);
stringstream ss(linha);
while (getline(ss, campo, ',')) {
    cout << campo << " | ";
}
```


UNIVALI

Desafio: matriz a partir de um arquivo

- A função abaixo lê uma matriz de um arquivo .txt e a carrega em uma matriz de caracteres:

```
void carrega_arquivo (char m[L][C]) {
    ifstream arquivo ("file1.txt");
    char letra;
    for (int i = 0; i < L; i++) {
        for (int j = 0; j < C+1; j++) { // após cada linha existe um \n
            arquivo.get(letra);
            if (j != C) { m[i][j] = letra; } // ignora o \n
        }
    }
}
```

Agora tente você: leia uma matriz a partir de um arquivo .csv, e salve os valores em uma matriz de inteiros.

UNIVALI

Modos de Arquivo Disponíveis

- ios_base::in — abre arquivo para leitura.
- ios_base::out — abre arquivo para escrita.
- ios_base::ate — procura o final do arquivo ao abri-lo.
- ios_base::app — anexa os dados a serem escritos ao final do arquivo.
- ios_base::trunc — trunca (apaga) os dados existentes no arquivo.
- ios_base::binary — abre e trabalha com arquivos em modo binário.

UNIVALI

Conectando com o que já vimos

- Structs e arquivos combinam muito bem: um arquivo é o jeito natural de guardar uma lista de registros — como um vetor de structs — de forma permanente.

```
struct Aluno { string nome; float nota; };

ofstream f;
f.open("alunos.txt");
f << aluno.nome << "," << aluno.nota << "\n";
```

- **Cada linha do arquivo vira um registro; cada campo separado por vírgula vira um membro da struct — é a mesma lógica de leitura por campos que acabamos de ver com CSV (stringstream + getline).**

UNIVALI

Resumo: usando arquivos

1. Inclua a biblioteca fstream.
2. Escolha a classe certa: ofstream (escrita), ifstream (leitura), ou fstream (os dois).
3. Sempre: crie o stream → open() → verifique com is_open() → leia/escreva → close().
4. Para ler um arquivo inteiro, prefira while (getline(...)) a checar eof() antes de ler.
5. Em CSV, cada vírgula separa um campo — use stringstream para extrair os valores de cada linha.

UNIVALI

Obrigado pela atenção

contato: Felski@univali.br